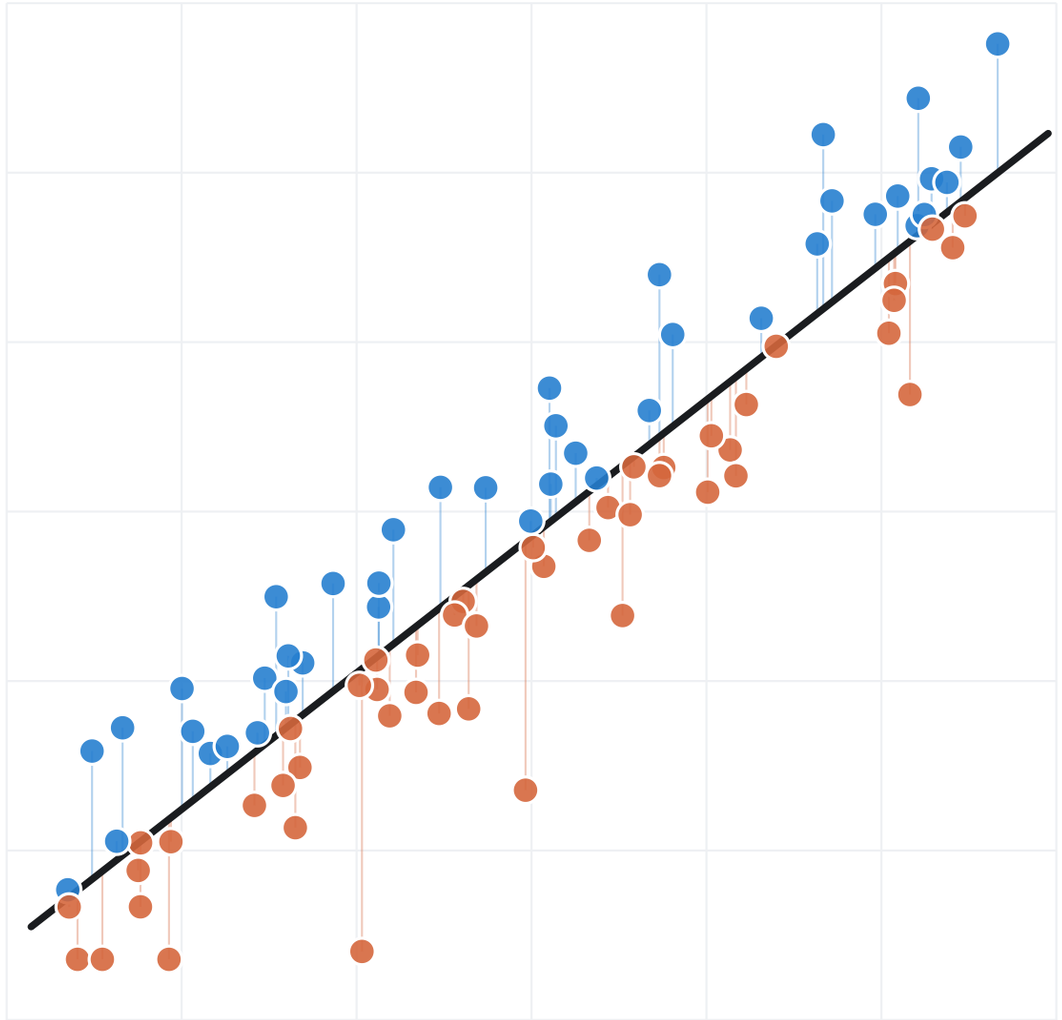
Линейные модели



Линейная и логистическая регрессия: от прямой по точкам до вероятности класса

Т-поколение ИИ · базовая ML-группа

15 ноября 2026

ЧАСТЬ 1

Линейная регрессия

Раньше прогноз собирали из деревьев. Теперь соберём его из признаков, умноженных на веса.

От суммы деревьев к сумме вкладов признаков

Меняем семейство моделей. Схема обучения остаётся знакомой.

БУСТИНГ

$$\hat{y}(x) = \hat{y}^{(0)} + \eta \sum_{m=1}^{M} h_m(x)$$

Сумма поправок от деревьев.

Обучаем: условия в узлах и ответы в листьях.

ЛИНЕЙНАЯ МОДЕЛЬ

$$\hat{y}(x) = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

Сумма вкладов признаков.

Обучаем: веса $w_1, \ldots, w_d$ и свободный член b.

Данные
train, validation, test

→

Прогноз
модель с параметрами

→

Ошибка
функция потерь

→

Обновление
меняем параметры

Четыре разных решения в одной задаче

Их часто путают. Каждое можно поменять, не трогая остальные.

Что выбираем	Вопрос	Пример сегодня
Модель	Как из признаков получается прогноз?	$\hat{y} = w \cdot x + b$
Функция потерь	Что минимизируем при обучении?	средний квадрат ошибки, MSE
Способ оптимизации	Как ищем минимум?	формула МНК или градиентный спуск
Метрика	Как оцениваем на отложенных данных?	RMSE на validation

ВОПРОС В ЗАЛ

Заменили при обучении MSE на MAE. Модель осталась линейной?

Датасет Auto MPG: экономичность автомобилей

398 автомобилей 1970–1982 годов. Задача: по характеристикам предсказать, сколько миль машина проезжает на галлоне топлива.

Столбец	Смысл
`mpg`	ответ: миль на галлон, больше — экономичнее
`weight`	масса, фунты
`horsepower`	мощность, л. с.; у 6 машин пропущена
`displacement`, `cylinders`	объём двигателя и число цилиндров
`acceleration`, `year`	время разгона и год модели
`origin`	категория: США, Европа, Япония

Разбиение

238 train — обучение.

80 validation — выбор настроек.

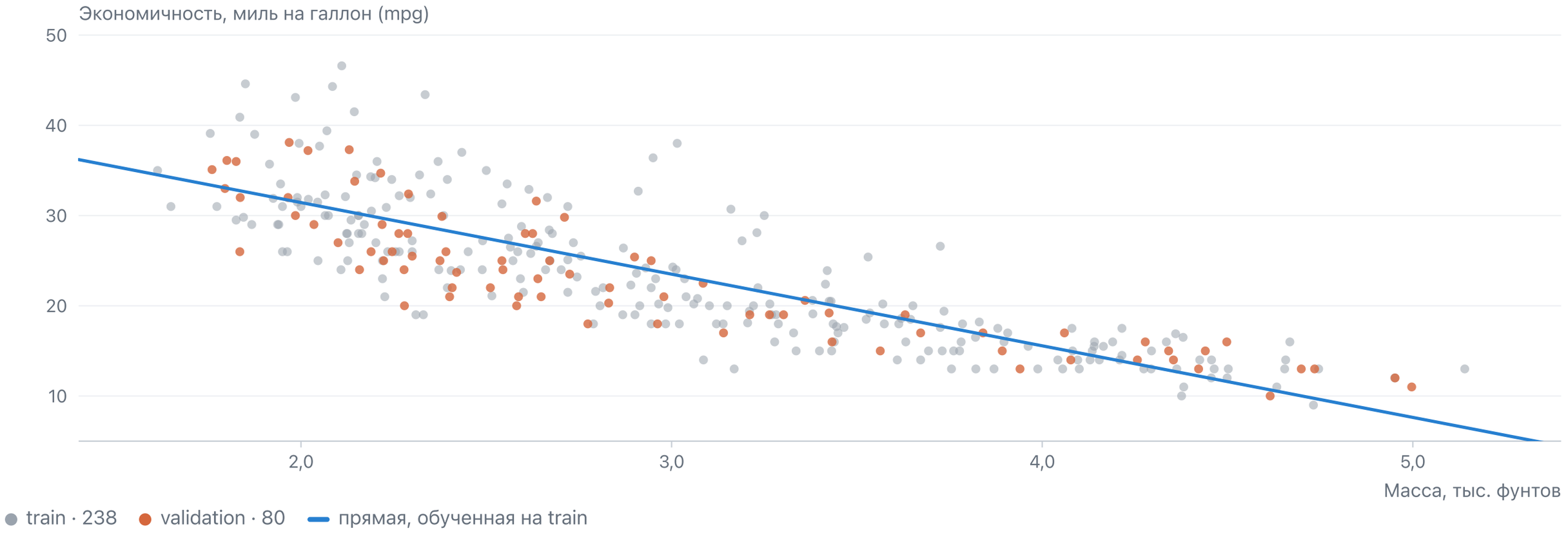
80 test — итоговая оценка, сегодня не трогаем.

Случайное разбиение 60 / 20 / 20, `random_state=42`.

Данные: UCI Machine Learning Repository · Auto MPG · R. Quinlan, 1993 · CC BY 4.0.

Один признак: прямая «масса → экономичность»

$$\hat{y} = 47{,}32 - 7{,}94 \cdot x \qquad x \text{ — масса в тысячах фунтов}, \quad \hat{y} \text{ — прогноз mpg}$$



RMSE на validation: **3,77 mpg**. Если всем предсказывать среднее по train, RMSE будет **7,08 mpg**.

Что означают наклон и свободный член

Наклон $w = -7{,}94$

Машина тяжелее на 1000 фунтов → прогноз меньше на 7,94 mpg.

Масса	Расчёт	Прогноз
3000 фунтов	47,32 – 7,94 · 3	23,51 mpg
4000 фунтов	47,32 – 7,94 · 4	15,57 mpg

У веса есть единицы: mpg на тысячу фунтов.

Свободный член $b = 47{,}32$

Прогноз при $x = 0$. Машин с нулевой массой нет: самая лёгкая в train весит 1613 фунтов.

Число b задаёт положение прямой. Отдельного смысла «экономичность невесомой машины» у него нет.

> Вес описывает **прогноз модели**. Он не доказывает, что облегчение конкретной машины даст именно такой прирост.

Единицы измерения меняют вес, но не прогноз

Запишем массу в фунтах вместо тысяч фунтов и обучим модель заново.

Единица массы	Вес w	Вклад для машины 3000 фунтов	Прогноз
тысячи фунтов	−7,939	$-7{,}939 \cdot 3 = -23{,}82$	23,51 mpg
фунты	−0,007939	$-0{,}007939 \cdot 3000 = -23{,}82$	23,51 mpg

Признак стал в 1000 раз больше — вес стал в 1000 раз меньше. Произведение то же.

По модулю веса нельзя сравнивать важность признаков в разных единицах.
К этому вернёмся, когда дойдём до стандартизации.

Несколько признаков: прогноз — сумма вкладов

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = w \cdot x + b$$

Модель на трёх признаках. Машина из validation: Dodge Aspen 1979 года, 3360 фунтов, 110 л. с.

Признак	Значение x	Вес w	Вклад w · x
Масса, тыс. фунтов	3,36	−6,41	−21,53
Мощность, сотни л. с.	1,10	−1,14	−1,25
Год модели минус 1970	9	+0,78	+7,02
Свободный член b			+39,13
Прогноз			**23,37 mpg**

Настоящий ответ — 20,6 mpg. RMSE на validation снизился с 3,77 до 2,95 mpg. Модель обучена на 233 машинах train с известной мощностью.

Вся выборка одной формулой

$$\hat{y} = Xw + b$$

Обозначение	Размер	Смысл
X	$n \times d$	строка — объект, столбец — признак
w	d	по одному весу на признак
b	число	прибавляется к каждому прогнозу
$\hat{y}$	n	прогнозы для всех объектов

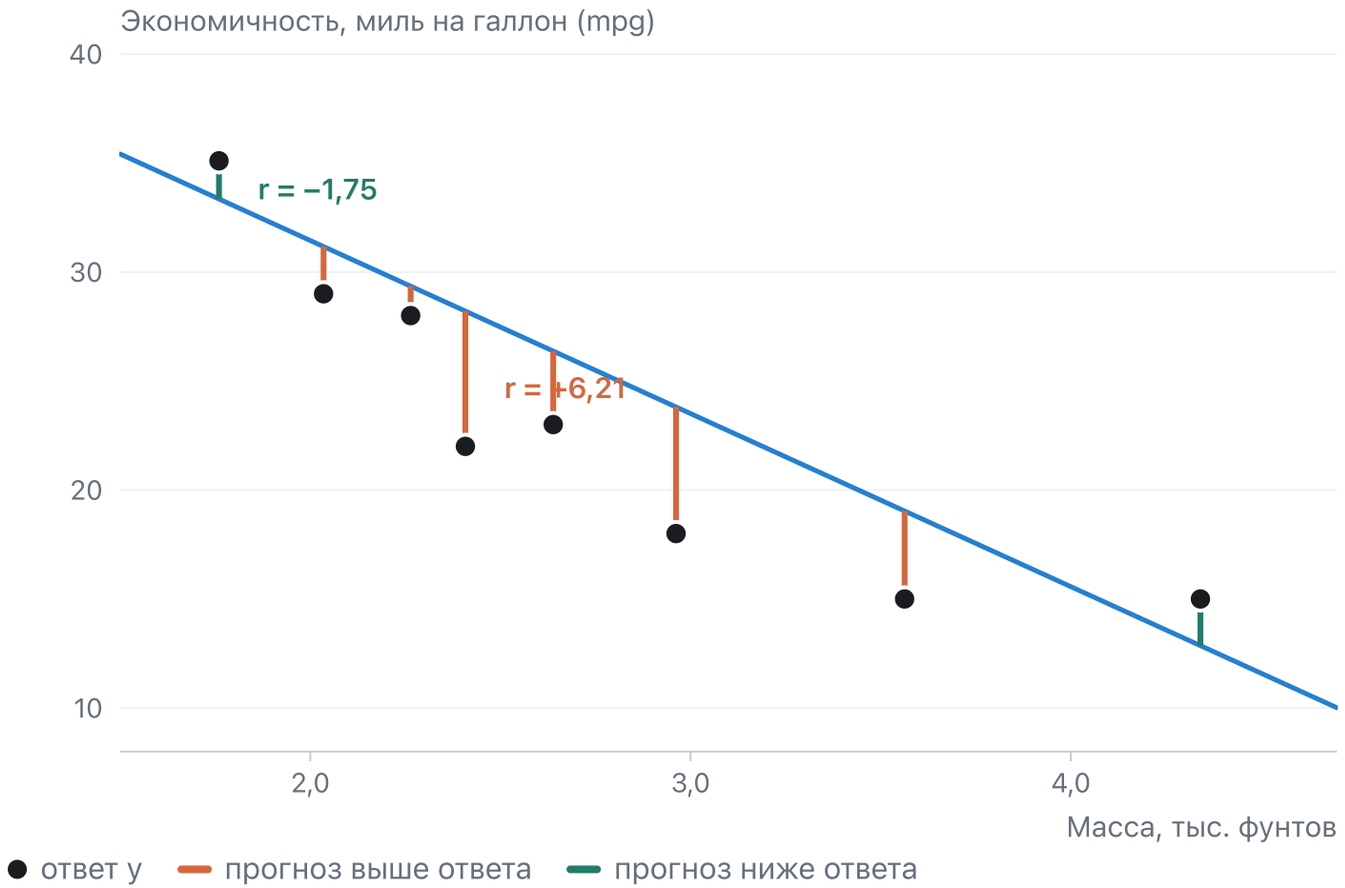
```
X.shape           # (233, 3)
w = np.array([-6.41, -1.14, 0.78])
b = 39.13

y_pred = X @ w + b
y_pred.shape      # (233,)
```

Оператор `@` из лекции по NumPy: каждая строка X скалярно умножается на w.

Геометрически: один признак — прямая, два — плоскость, больше — гиперплоскость.

Остаток — на сколько прогноз разошёлся с ответом



$$r_i = \hat{y}_i - y_i$$

r > 0 — модель завысила прогноз.

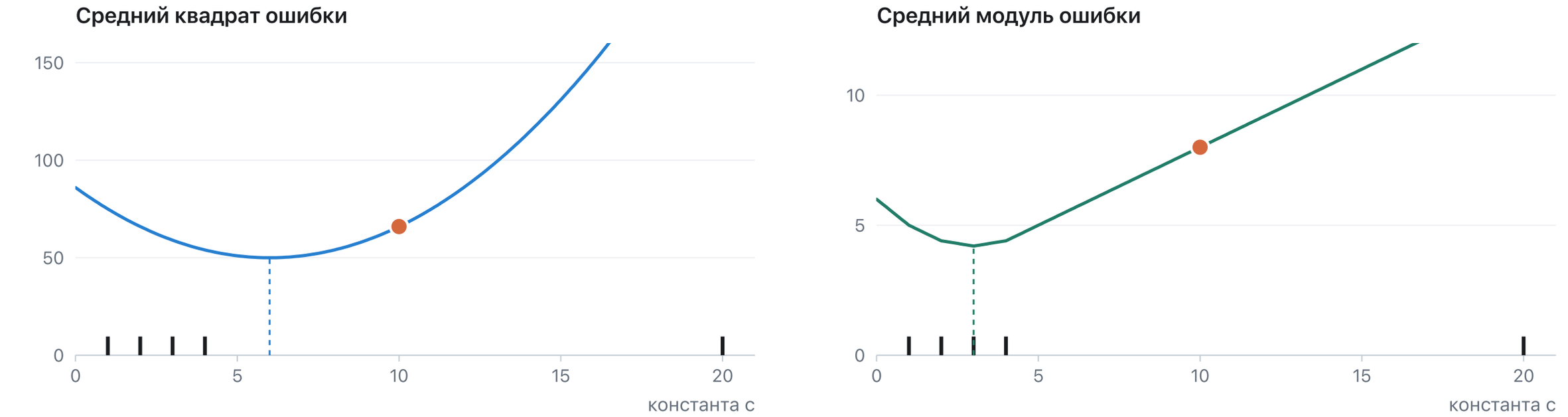
r < 0 — занизила.

Машина	y	ŷ	r
Honda Civic 1300	35,1	33,35	−1,75
Chevrolet Vega (sw)	22,0	28,21	+6,21

На графике восемь машин из validation. Сегодня остаток всегда «прогноз минус ответ».

Сначала проще: какую одну константу предсказать всем?

Ответы: 1, 2, 3, 4, 20. Константа c **10,0**



c = **10,0** · средний квадрат ошибки **66,0** (минимум 50,0 при c = 6, это среднее) · средний модуль **8,0** (минимум 4,2 при c = 3, это медиана)

Функция потерь определяет ответ: квадраты ошибок выбирают **среднее**, модули — **медиану**.
Выброс 20 тянет среднее сильнее: его ошибка входит в сумму в квадрате.

Метод наименьших квадратов

Ищем веса, при которых средний квадрат остатка на train минимален.

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^{n} \underbrace{\left(w\, x_i + b - y_i \right)}_{r_i}^2 \longrightarrow \min_{w,\, b}$$

Почему квадрат

Знак ошибки не важен. Большие промахи штрафуются сильнее.

Зачем 1/2

Сократится с двойкой при взятии производной. Положение минимума не меняет.

Связь с метриками

$J = \frac{1}{2}\,\mathrm{MSE},\ \mathrm{RMSE} = \sqrt{\mathrm{MSE}}$. Минимум у всех трёх — при одних и тех же весах.

J — парабола по w и по b. В минимуме обе производные равны нулю — как на лекции про оптимизацию.

Шаг 1. Производная по b

$$\frac{\partial J}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}\left(w\,x_i + b - y_i\right) = w\,\bar{x} + b - \bar{y} = 0$$

$$b = \bar{y} - w\,\bar{x}$$

Что это значит

Лучшая прямая проходит через точку средних $(\bar{x},\ \bar{y})$.

Сумма остатков на train равна нулю: модель в среднем не завышает и не занижает.

Проверка на Auto MPG

$\bar{x} = 3{,}019$ тыс. фунтов, $\bar{y} = 23{,}35$ mpg.

$47{,}32 - 7{,}94 \cdot 3{,}019 = 23{,}35$ ✓

Шаг 2. Производная по w

Подставим $b = \bar{y} - w\bar{x}$: остаток станет $r_i = w(x_i - \bar{x}) - (y_i - \bar{y})$. Приравняем производную к нулю:

$$\frac{\partial J}{\partial w} = \frac{1}{n}\sum_{i=1}^{n}\big[w(x_i - \bar{x}) - (y_i - \bar{y})\big](x_i - \bar{x}) = 0$$

$$w = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

Числитель: меняются ли x и y согласованно. Оба выше среднего или оба ниже — слагаемое положительное.

Знаменатель: разброс x. Если все x_i одинаковые, он равен нулю — наклон по таким данным определить нельзя.

МНК руками: три точки

Точки (0; 1), (1; 2), (2; 2). Средние: $\bar{x} = 1, \bar{y} = \frac{5}{3}$.

x	y	$x - \bar{x}$	$y - \bar{y}$	произведение
0	1	−1	−2/3	2/3
1	2	0	1/3	0
2	2	1	1/3	1/3
Сумма		$\sum(x - \bar{x})^2 = 2$		**1**

$$w = \frac{1}{2}, \qquad b = \frac{5}{3} - \frac{1}{2} \cdot 1 = \frac{7}{6}$$

x	y	$\hat{y}$	r = $\hat{y}$ − y
0	1	7/6	+1/6
1	2	5/3	−1/3
2	2	13/6	+1/6

Сумма остатков: 1/6 − 1/3 + 1/6 = 0. Этот пример пересчитаем на семинаре в NumPy.

Много признаков: та же идея в матричной записи

Допишем к X столбец единиц: $\tilde{X} = [\,1,\ X\,], \theta = (b,\ w)$. Тогда $\hat{y} = \tilde{X}\theta$, и все производные равны нулю, когда

$$\tilde{X}^\top(\tilde{X}\theta - y) = 0 \qquad \Longrightarrow \qquad \theta = (\tilde{X}^\top\tilde{X})^{-1}\tilde{X}^\top y$$

Формула с обратной матрицей работает, когда столбцы $\tilde{X}$ не выражаются друг через друга. В коде матрицу не обращают — вызывают устойчивый решатель.

```
X1 = np.c_[np.ones(len(X)), X]
theta, *_ = np.linalg.lstsq(X1, y, rcond=None)

# то же самое в scikit-learn
model = LinearRegression().fit(X, y)
model.intercept_, model.coef_
```

Одна формула вместо перебора: МНК решается точно и быстро.

Другой путь к тому же минимуму: градиентный спуск

Производная по каждому весу — среднее произведение остатка на признак.

$$\frac{\partial J}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n} r_i\, x_{ij} \qquad \frac{\partial J}{\partial b} = \frac{1}{n}\sum_{i=1}^{n} r_i$$

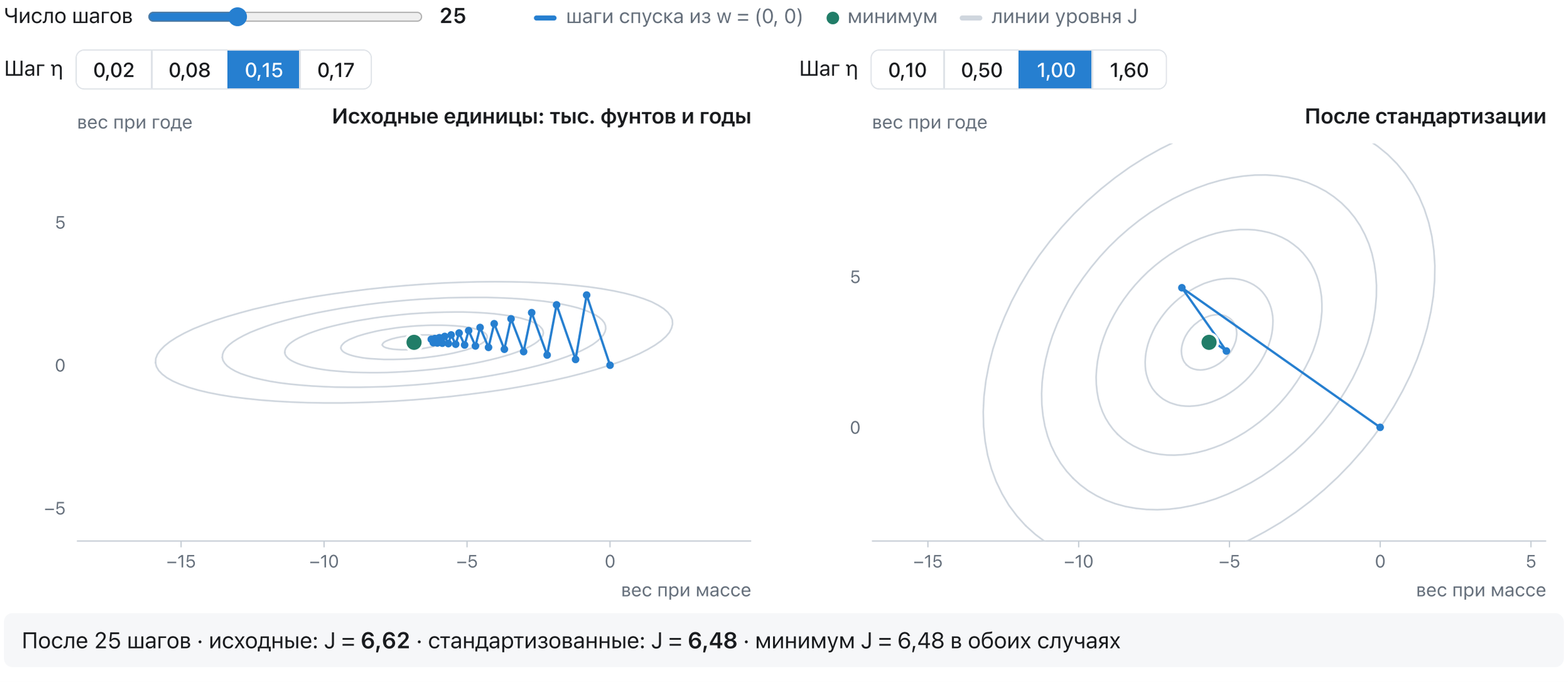
$$\nabla_w J = \frac{1}{n} X^\top r$$

```
w, b = np.zeros(d), 0.0
for step in range(n_steps):
    r = X @ w + b - y           # остатки
    w -= eta * X.T @ r / len(y)
    b -= eta * r.mean()
```

η — размер шага, как `learning_rate` в бустинге.

> Объект сильнее двигает вес, если у него большая ошибка **и** большое значение признака.

Масштаб признаков меняет путь градиентного спуска



Auto MPG, train: предсказываем mpg по массе и году модели, признаки и ответ центрированы. Вертикальная ось — вес при годе модели. Масштабы осей на каждом графике равны.

МНК — цель, градиентный спуск — способ

ТОЧНОЕ РЕШЕНИЕ

`LinearRegression`

Решает систему уравнений за один вызов.

Нет шага и числа итераций. Масштаб признаков на прогнозы не влияет.

Подходит, пока таблица помещается в память.

ИТЕРАЦИИ

Градиентный спуск, `SGDRegressor`

Двигается к минимуму небольшими шагами.

Нужны шаг η и стандартизация признаков.

Работает на очень больших данных и с любой дифференцируемой потерей — так обучается логрег и нейросети.

Если спуск сошёлся, прогнозы совпадают с точным решением: минимизируем одну и ту же функцию J.

БЫСТРЫЙ ОПРОС

Массу перевели из тысяч фунтов в килограммы и заново обучили МНК на том же train. Что изменится?

A Прогнозы и RMSE на validation

B Ничего: вес останется −7,94

✓ Вес при массе изменится, прогнозы останутся теми же

D Модель перестанет быть линейной

Умножение признака на константу МНК компенсирует весом: w · x не меняется. У точного решения без штрафа прогнозы от единиц измерения не зависят.

БЫСТРЫЙ ОПРОС

Градиентный спуск для линейной регрессии расходится: J растёт с каждым шагом. Что пробуем первым?

A Добавить больше итераций

✓ Уменьшить шаг η и стандартизовать признаки

C Заменить MSE на RMSE

D Убрать свободный член b

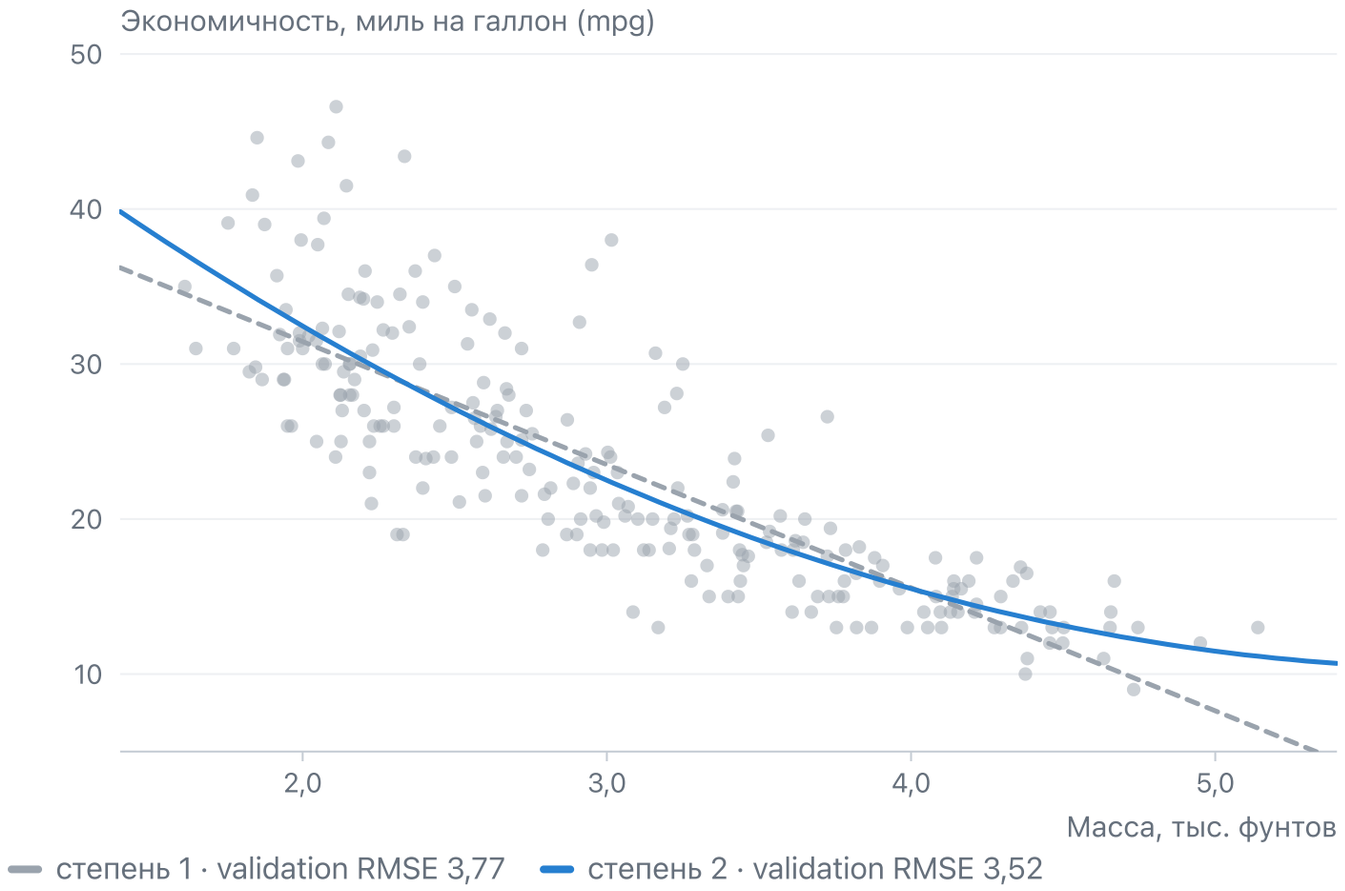
Расходимость — признак слишком большого шага для самого «крутого» направления. Стандартизация выравнивает масштабы, после неё один шаг подходит всем весам.

ЧАСТЬ 2

Признаки, масштаб и штраф

Линейная модель проста. Её возможности определяют признаки, которые мы ей дали, и ограничения на веса.

Линейная модель умеет рисовать кривые



Добавим в таблицу новый столбец — квадрат массы:

$$\hat{y} = 61{,}22 - 17{,}34\,x + 1{,}48\,x^2$$

График — парабола. Но модель по-прежнему **линейна по весам**: прогноз — сумма «вес × столбец».

Алгоритм обучения тот же — МНК. Изменилась только матрица признаков: было $[x]$, стало $[x,\ x^2]$.

«Линейная» — про то, как в формулу входят веса, а не про форму графика.

Какие признаки можно добавить

$$x \longrightarrow \varphi(x) \longrightarrow \hat{y} = w \cdot \varphi(x) + b$$

Степени и функции

$x^2,\ \sqrt{x},\ \ln x$

Изгиб зависимости. Логарифм — только для положительных значений.

Взаимодействия

$x_1 \cdot x_2$

Эффект одного признака зависит от другого. Без такого столбца вклады просто складываются.

Категории

one-hot: столбцы из 0 и 1

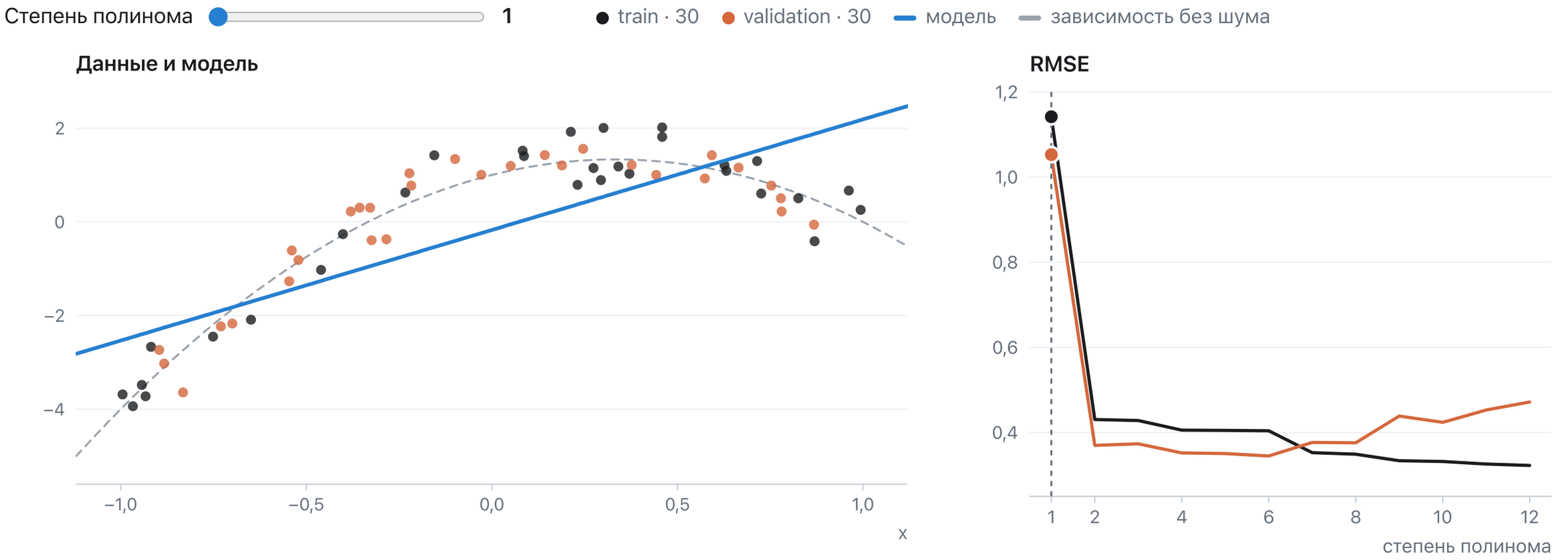
У каждой категории свой сдвиг прогноза.

Пример one-hot. Происхождение машины: США — опорная категория, для Европы и Японии — по столбцу.

$$\hat{y} = 43{,}64 - 7{,}03 \cdot \text{масса} + 2{,}41 \cdot [\text{Европа}] + 2{,}85 \cdot [\text{Япония}]$$

При той же массе прогноз для японской машины на 2,85 mpg выше, чем для американской. На validation RMSE почти не изменился: 3,78 против 3,77.

Больше признаков — больше возможностей и больше риска



Степень **1** · признаков: 1 · RMSE train **1,142** · RMSE validation **1,053** · минимум validation: степень 6 (0,345), у степени 2 — 0,370

Учебная синтетика: $y = 1 + 2x - 3x^2 + \text{шум } N(0; 0{,}4^2)$, $x \sim U(-1; 1)$, генератор NumPy с seed = 0. Модель обучается только на train.

Нормализация и стандартизация: один масштаб для всех признаков

Меняем масштаб каждого признака. Порядок объектов по признаку остаётся прежним.

Нормализация · `MinMaxScaler`

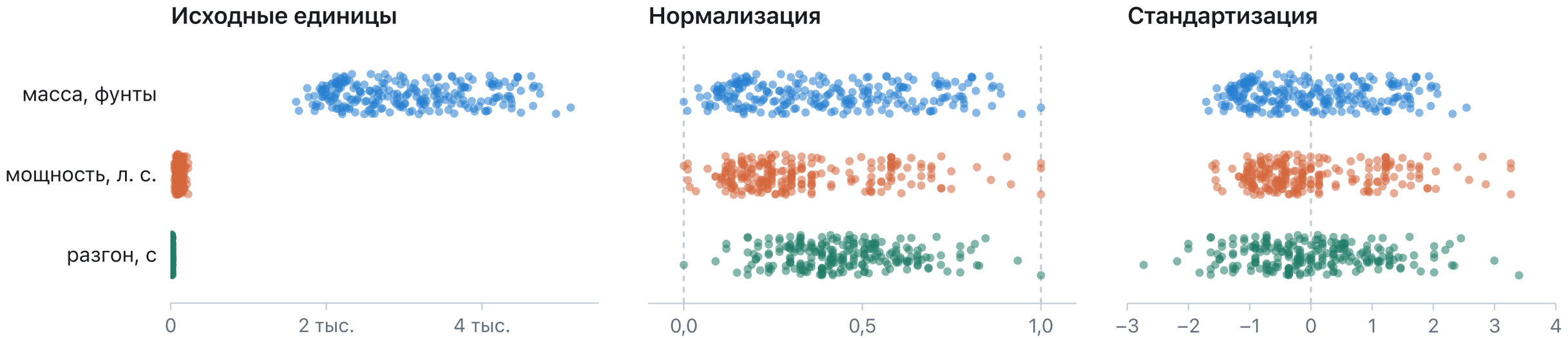
$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0;\ 1]$$

Один выброс сожмёт остальные значения к краю.

Стандартизация · `StandardScaler`

$$x' = \frac{x - \bar{x}}{s_x}: \quad \text{среднее } 0,\ \text{разброс } 1$$

Основной выбор для линейных моделей.



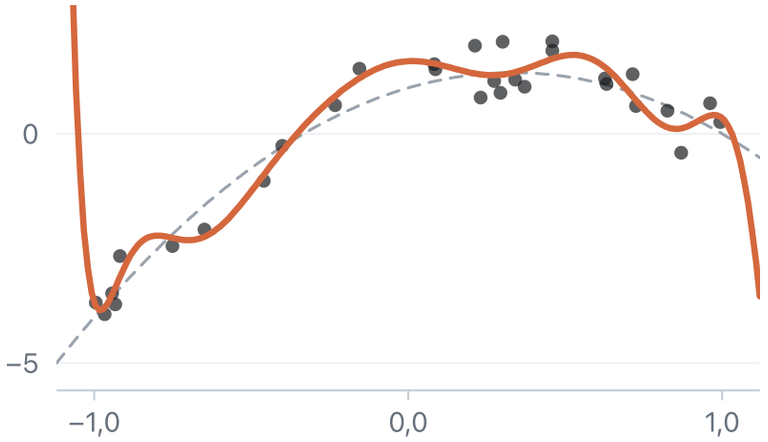
Auto MPG, 233 машины train с известной мощностью. На общей оси в исходных единицах мощность и разгон почти не видны: их числа в десятки раз меньше массы. Dodge Aspen, 3360 фунтов: нормализация (3,36 – 1,61) / (5,14 – 1,61) = 0,50, стандартизация (3,36 – 3,02) / 0,83 = 0,41.

Регуляризация: штраф за большие веса

$$J(w, b) + \frac{\lambda}{2} \sum_j w_j^2$$

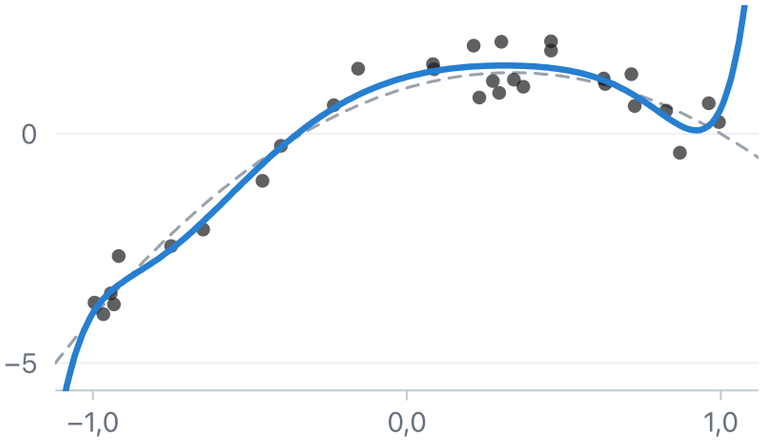
Меняем не данные, а **функцию потерь**: большие веса теперь стоят дорого. Пример — полином степени 12 из опыта со степенями.

Без штрафа



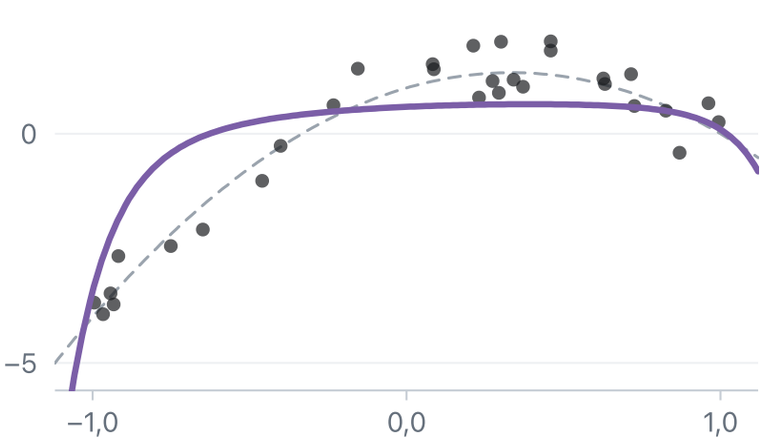
‖w‖ = **275** · RMSE validation **0,47**

Подобранный штраф, alpha = 0,03



‖w‖ = **3,1** · RMSE validation **0,33**

Сильный штраф, alpha = 100



‖w‖ = **0,4** · RMSE validation **0,98**

Чёрные точки — train, пунктир — зависимость без шума. Степени x, …, x^{12} стандартизованы по train; подобранный alpha — лучший по validation из 15 значений от 0,0001 до 1000.

Нужна середина между «цепляется за шум» и «всё выпрямил» — её ищем по validation.

Когда масштаб признаков важен

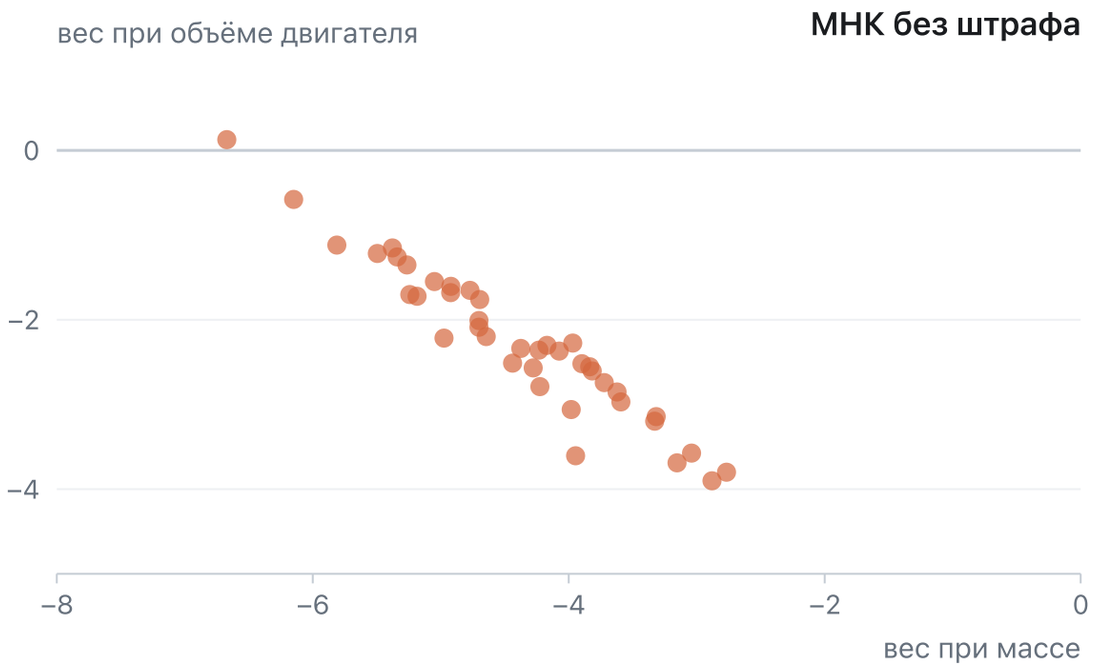
Модель или метод	Нужно масштабировать?	Почему
Дерево, случайный лес, бустинг	**нет**	условие «x ≤ порог» не зависит от единиц
МНК без штрафа, точное решение	**нет**	вес компенсирует единицы, прогнозы те же
Градиентный спуск	**да**	один шаг η на все веса
Ridge, Lasso, логрег со штрафом	**да**	штраф сравнивает веса между собой
kNN, k-means	**да**	расстояние складывает разности признаков

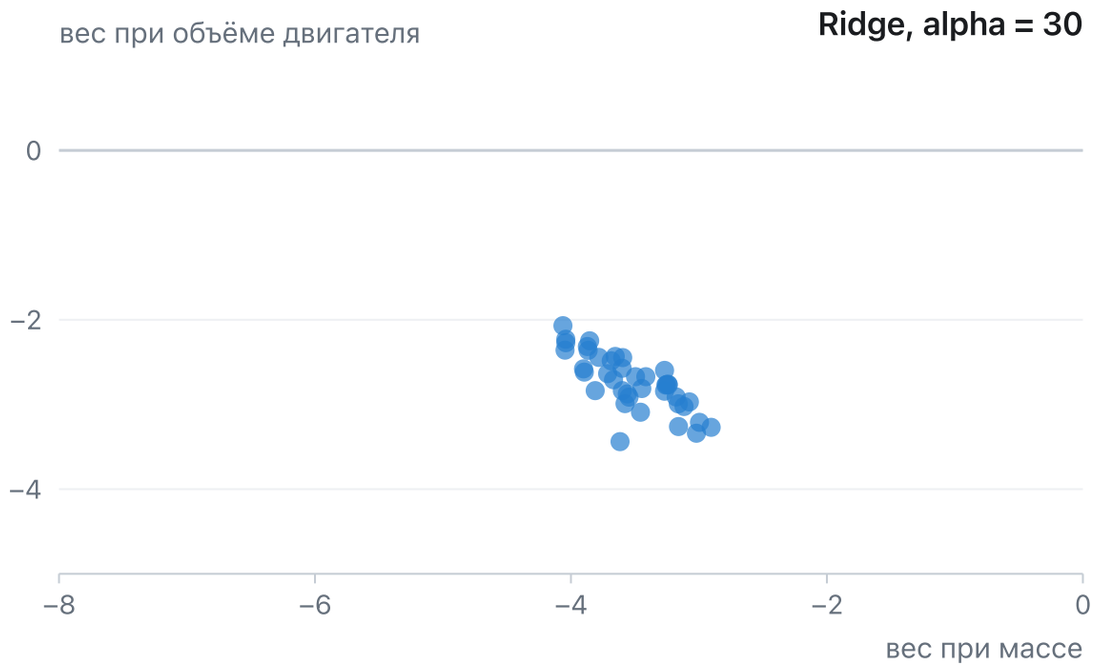
```
model = make_pipeline(
    StandardScaler(),
    Ridge(alpha=10),
)
model.fit(X_train, y_train)
model.predict(X_val)
```

Среднее и разброс считаем **только по train**. `Pipeline` делает это сам: при `fit` обучает scaler, при `predict` применяет готовый.

Похожие признаки: веса скачут, прогноз почти не меняется

Масса и объём двигателя почти дублируют друг друга. Обучим модель 40 раз на случайных подвыборках train.





Разброс весов по 40 бутстреп-выборкам train: МНК — **0,88**, Ridge — **0,32**. Разброс прогнозов на validation: **0,45** и **0,35** mpg. RMSE validation: 3,77 и 3,68 mpg.

Auto MPG · признаки стандартизованы по train · корреляция массы и объёма на train: 0,92 · alpha = 30 задан заранее для демонстрации, без подбора.

Регуляризация: добавим цену за большие веса

L2-ШТРАФ

Ridge

$$J(w, b) + \frac{\lambda}{2} \sum_{j=1}^{d} w_j^2$$

Плавно уменьшает все веса. Похожие признаки делят вклад между собой.

L1-ШТРАФ

Lasso

$$J(w, b) + \lambda \sum_{j=1}^{d} |w_j|$$

Часть весов делает точно нулевыми — признаки выпадают из модели.

λ Сила штрафа, $\lambda \geq 0$. При $\lambda = 0$ получаем обычный МНК. Слишком большая λ — все веса около нуля, модель недообучена.

b Свободный член не штрафуем: сдвиг всех прогнозов не делает модель сложнее.

Та же идея, что ограничение глубины и шага в бустинге: проигрываем немного на train, чтобы выиграть на новых данных.

Как штраф действует на один вес

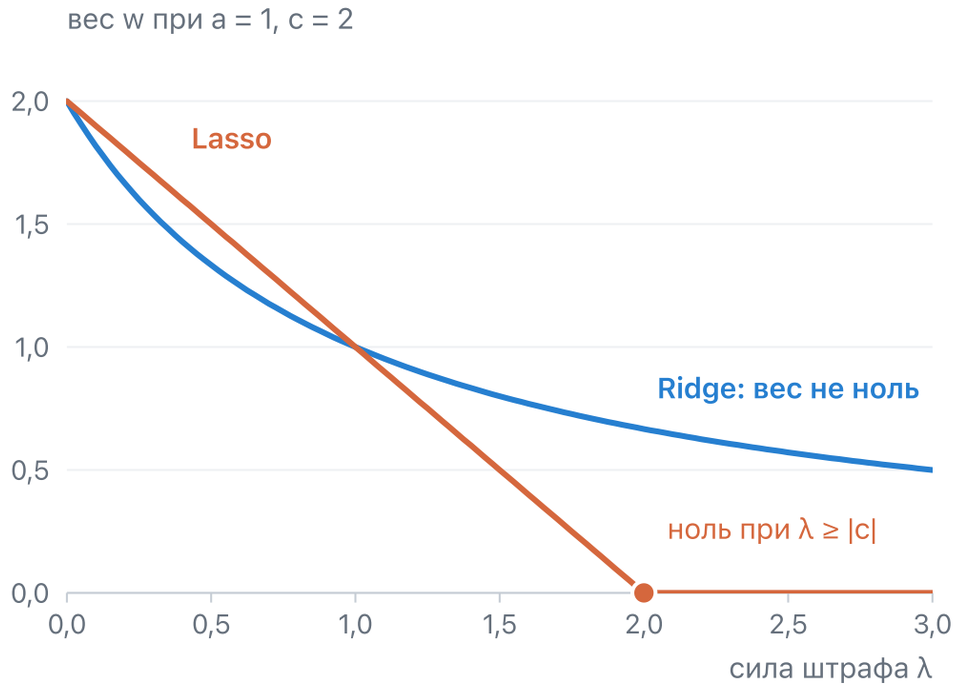
Один центрированный признак. Обозначим $a = \frac{1}{n}\sum x_i^2$ и $c = \frac{1}{n}\sum x_i y_i$. Тогда МНК даёт $w = c/a$.

Ridge: $(a+\lambda)\,w - c = 0 \implies w = \dfrac{c}{a+\lambda}$

Знаменатель растёт — вес сжимается, но нулём не становится.

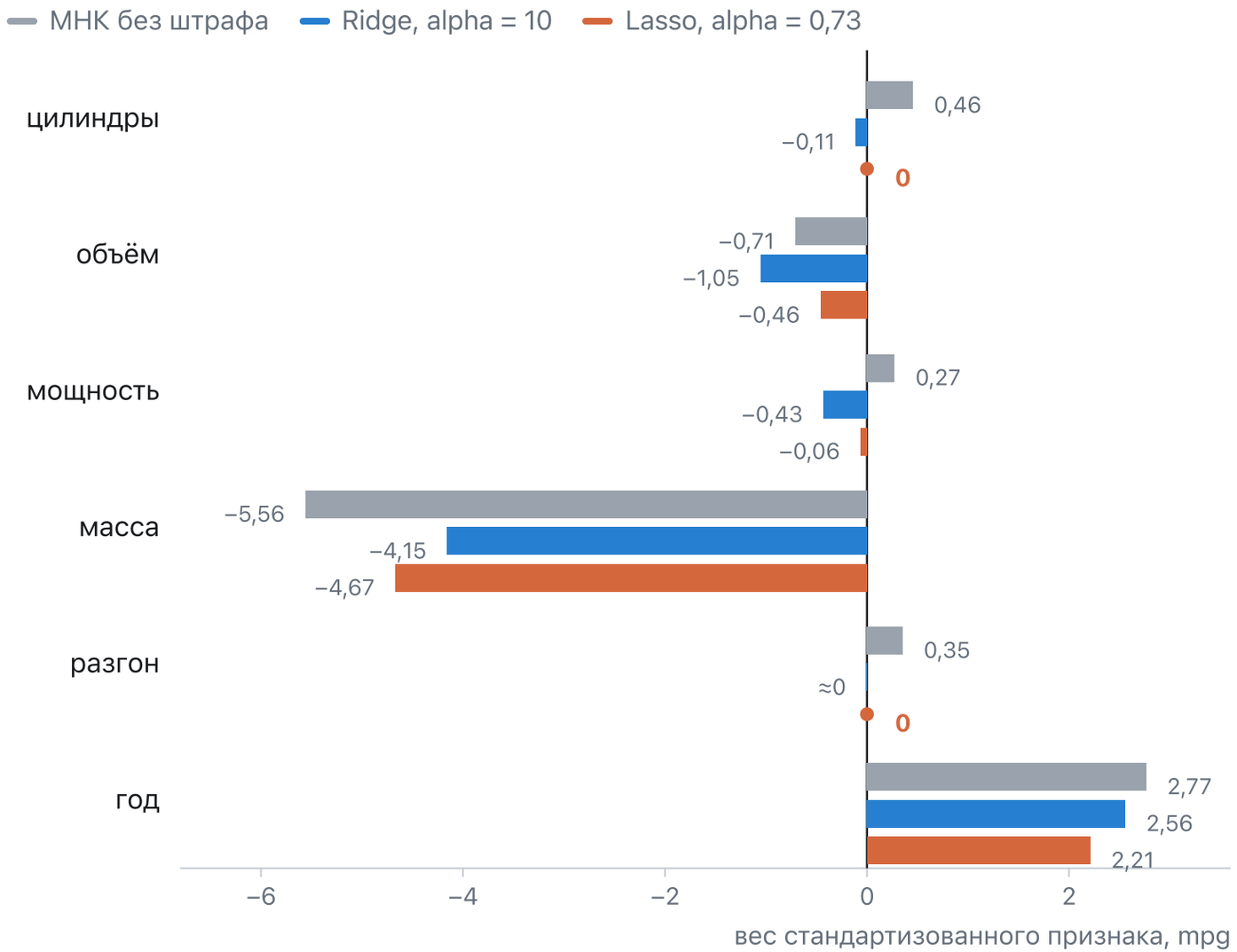
Lasso: $w = \operatorname{sign}(c)\cdot\dfrac{\max(|c|-\lambda,\ 0)}{a}$

Из $|c|$ вычитается λ. Если связь признака с ответом слабее штрафа, вес — точный ноль.



Для нескольких коррелирующих признаков эти формулы по отдельности не применяются — там веса влияют друг на друга.

Ridge и Lasso на Auto MPG: что стало с весами



Шесть признаков, все стандартизованы. Цилиндры, объём, мощность и масса сильно связаны.

МНК. Вес мощности **положительный** — будто мощная машина экономичнее. Так модель компенсирует похожие признаки.

Ridge. Все веса меньше по модулю, знак мощности стал естественным. Нулей нет.

Lasso. Цилиндры и разгон — **ровно ноль**: модели хватило массы и объёма.

RMSE на validation: МНК 3,02 · Ridge 3,01 · Lasso 2,79 mpg. alpha у каждой модели выбран по validation.

Силу штрафа выбираем по validation

1. Разбить данные до любой подгонки.
2. Для каждого значения из сетки обучить `Pipeline` на train.
3. Выбрать значение с лучшей метрикой на validation.
4. Test — один раз, в самом конце.



При маленьком alpha штраф почти не действует: RMSE как у МНК без штрафа (3,02). Auto MPG, 6 признаков.

Как λ называется в scikit-learn

Класс	Параметр	Больше значение —
`Ridge`	`alpha`	сильнее штраф
`Lasso`	`alpha`	сильнее штраф
`LogisticRegression`	`C`	слабее штраф

В наших обозначениях: у `Lasso` $\text{alpha} = \lambda$, у `Ridge` $\text{alpha} = n\lambda$, у логрега $C = 1/(n\lambda)$. Библиотеки по-разному нормируют сумму ошибок, поэтому числа между классами не переносятся.

Результат одного разбиения — не рейтинг методов. На другой выборке порядок может быть иным.

БЫСТРЫЙ ОПРОС

Как правильно применить StandardScaler?

A Обучить на всей таблице, потом разбить на train и validation

✓ Обучить на train, применить к train, validation и test

C Обучить отдельно на train, отдельно на validation и отдельно на test

D Масштабировать только ответ у

Среднее и разброс — тоже параметры, выученные по данным. Если считать их по всей таблице, информация из validation и test утекает в обучение. Pipeline исключает эту ошибку.

БЫСТРЫЙ ОПРОС

Lasso обнулил вес признака «число цилиндров» и оставил «массу». Какой вывод верен?

A Число цилиндров не связано с экономичностью

B Масса — причина расхода, цилиндры — нет

C В данных ошибка, признак надо удалить из таблицы

✓ При этом штрафе модели хватило других признаков, похожих на этот

Цилиндры сильно коррелируют с массой и объёмом. Lasso оставляет один признак из группы похожих; какой именно — может зависеть от выборки. О причинности веса не говорят.

ЧАСТЬ 3

Логистическая регрессия

Несмотря на название, это классификатор. От линейной регрессии остаётся score $w \cdot x + b$, меняется то, что мы с ним делаем.

Что меняется при переходе к классификации

Два класса: y = 0 или y = 1. Хотим не только метку, но и уверенность.

	Линейная регрессия	Логистическая регрессия
Что считаем	$\hat{y} = w \cdot x + b$	score $z = w \cdot x + b$
Что выдаём	число $\hat{y}$	вероятность $p = \sigma(z)$, затем класс по порогу
Что минимизируем	квадрат ошибки	log loss
Градиент по score	$\hat{y} - y$	$p - y$

ВОПРОС В ЗАЛ

Почему нельзя просто обучить линейную регрессию на метках 0 и 1 и назвать прогноз вероятностью?

Шансы: как перейти от вероятности к числу без границ

Вероятность

$p \in (0;\ 1)$

p = 0,8

→

Шансы

$\frac{p}{1-p} \in (0;\ \infty)$

0,8 / 0,2 = 4, «четыре к одному»

→

Логарифм шансов

$\ln \frac{p}{1-p} \in (-\infty;\ \infty)$

ln 4 ≈ 1,39

Логарифм шансов принимает любые значения — как линейное выражение. Логистическая регрессия приравнивает их:

$$\ln \frac{p}{1-p} = w \cdot x + b$$

p = 0,5 ↔ шансы 1 ↔ логарифм шансов 0. Вероятности ниже 0,5 дают отрицательный score.

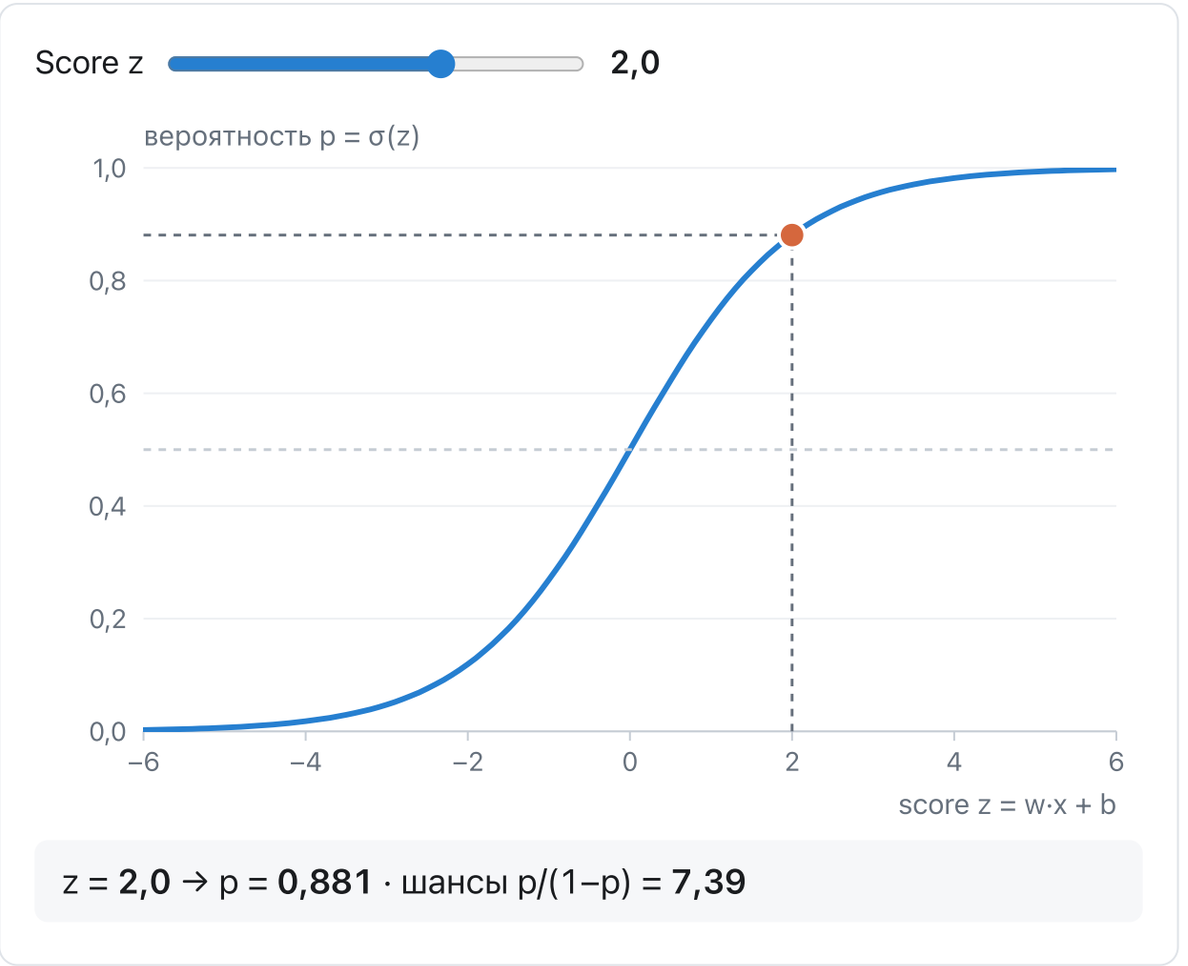
Сигмоида: обратно от score к вероятности

Выразим p из равенства $\ln \frac{p}{1-p} = z$:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

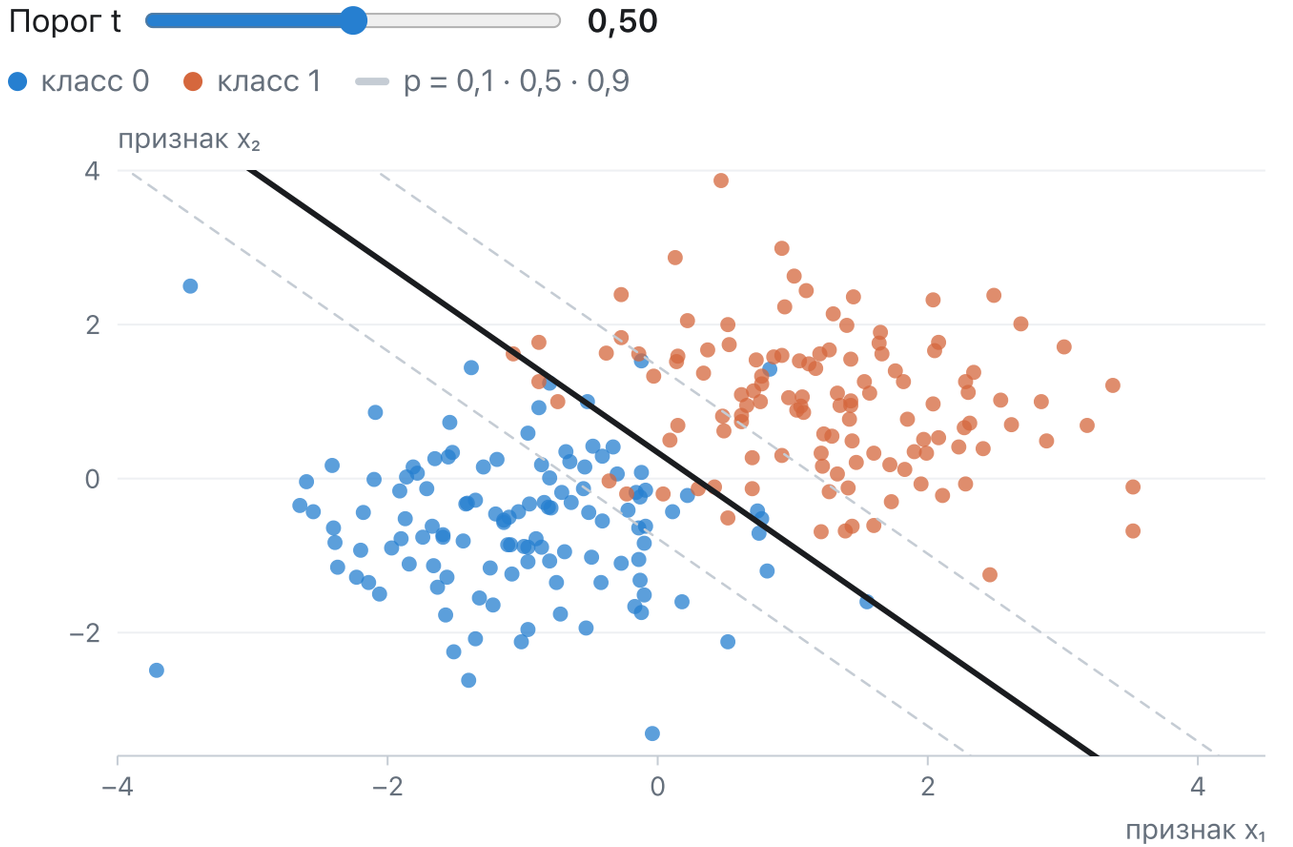
score z	шансы	p
−2	0,14	0,12
0	1	0,50
+2	7,39	0,88

Признак вырос на 1 → score вырос на w_j → шансы умножились на e^{w_j}. Вероятность при этом меняется не на постоянную величину.



z = **2,0** → p = **0,881** · шансы p/(1−p) = **7,39**

Порог превращает вероятность в класс



Порог t = **0,50** · граница: w·x + b = ln(t/(1–t)) = **0,00** · синих в классе 1: **5** · оранжевых в классе 0: **8**

Учебная синтетика: два нормальных облака по 120 точек, seed = 1; логрег обучен на показанных точках.

$$\hat{y} = \begin{cases} 1, & p \geq t \\ 0, & p < t \end{cases}$$

Условие $p \geq t$ — то же, что

$$w \cdot x + b \;\geq\; \ln \frac{t}{1-t}$$

Граница между классами — **прямая**. При $t = 0{,}5$ справа стоит 0.

Смена порога сдвигает границу параллельно. Веса модели при этом не меняются.

Как обучать: вероятность правильного ответа

Хорошая модель даёт высокую вероятность тому классу, который был на самом деле.

1 Вероятность наблюдаемого ответа для одного объекта: p_i, если $y_i = 1$, и $1 - p_i$, если $y_i = 0$. Одной формулой:

$$P(y_i \mid x_i) = p_i^{y_i}(1 - p_i)^{1 - y_i}$$

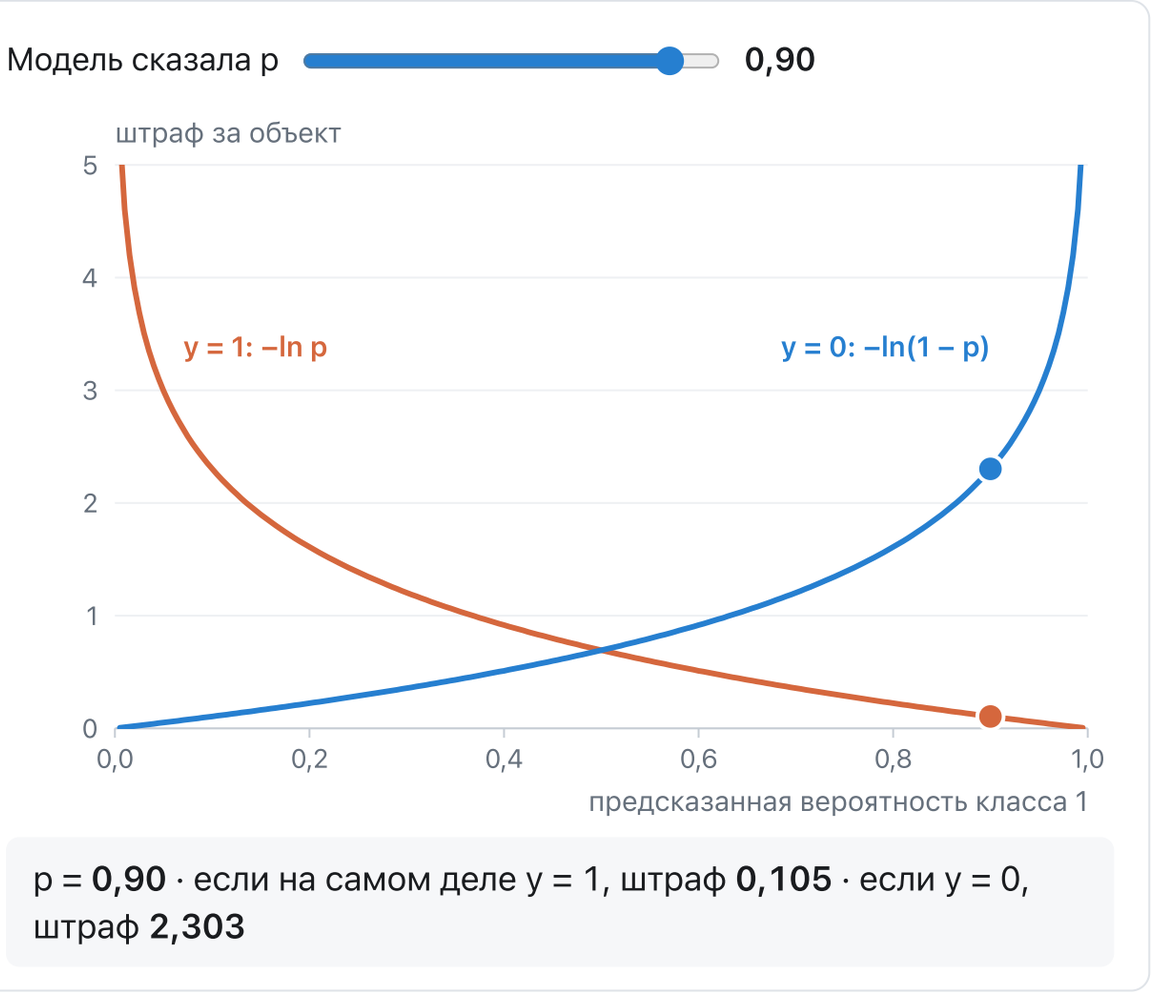
2 Для всей выборки вероятности перемножаются. Хотим максимум этого произведения — правдоподобия.

3 Логарифм превращает произведение в сумму. Минус и деление на n — получаем функцию потерь для минимизации.

$$J = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \ln p_i + (1 - y_i)\ln(1 - p_i)\Big]$$

Это **log loss**, он же логистическая функция потерь и бинарная кросс-энтропия.

Уверенная ошибка стоит дорого



p = **0,90** · если на самом деле y = 1, штраф **0,105** · если y = 0, штраф **2,303**

Штраф за один объект класса 1 равен $-\ln p$.

Модель сказала p	Штраф, если y = 1
0,99	0,010
0,90	0,105
0,50	0,693
0,10	2,303
0,01	4,605

«Не знаю» (p = 0,5) стоит 0,693. Уверенный промах (p = 0,01) — почти в семь раз дороже.

Log loss оценивает сами вероятности, а не только то, по какую сторону порога они оказались.

Градиент логрега: снова «прогноз минус ответ»

Для одного объекта: $\ell = -y \ln p - (1-y)\ln(1-p)$, где $p = \sigma(z)$. Правило цепочки:

$$\frac{\partial \ell}{\partial z} = \underbrace{\left(-\frac{y}{p} + \frac{1-y}{1-p}\right)}_{\partial \ell / \partial p} \cdot \underbrace{p(1-p)}_{\partial p / \partial z} = p - y$$

$$\nabla_w J = \frac{1}{n} X^\top (p - y)$$

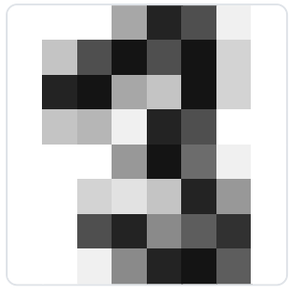
Сравните с линейной регрессией: $\nabla_w J = \frac{1}{n} X^\top (\hat{y} - y)$. Код спуска отличается одной строкой.

```
z = X @ w + b
p = 1 / (1 + np.exp(-z))      # новая строка
w -= eta * X.T @ (p - y) / len(y)
b -= eta * (p - y).mean()
```

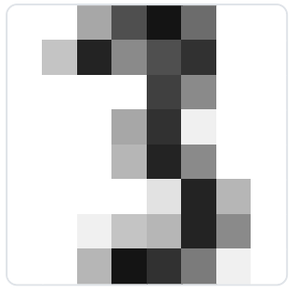
> В бустинге для классификации новое дерево обучали на $y - p$. Это тот же градиент с обратным знаком.

Пример: отличить цифру 3 от цифры 8

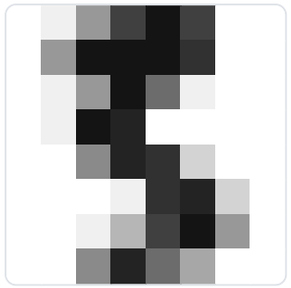
Датасет Digits: изображения 8×8, каждый пиксель — признак. Всего 64 признака, класс 1 — «восьмёрка».



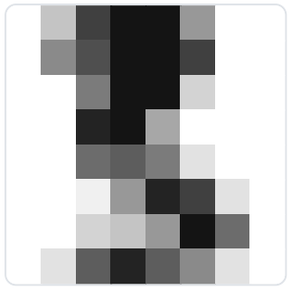
ответ: **3**
z = −2,8
p(8) = **0,058**
верно



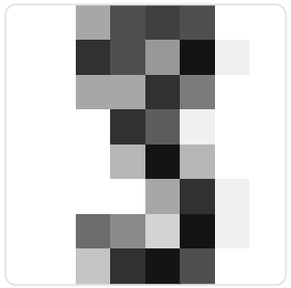
ответ: **3**
z = −13,2
p(8) = **0,000**
верно



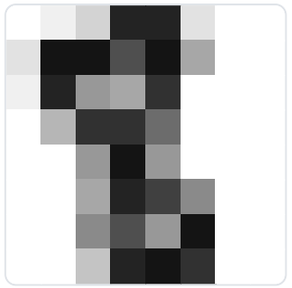
ответ: **3**
z = 0,8
p(8) = **0,693**
ошибка



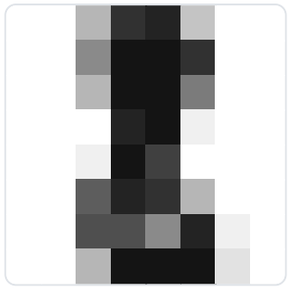
ответ: **3**
z = −1,1
p(8) = **0,258**
верно



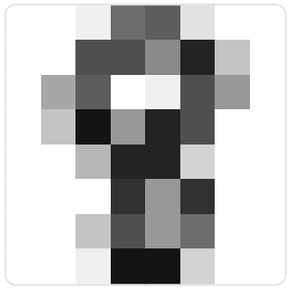
ответ: **3**
z = −1,1
p(8) = **0,250**
верно



ответ: **8**
z = 7,9
p(8) = **1,000**
верно



ответ: **8**
z = 5,4
p(8) = **0,995**
верно



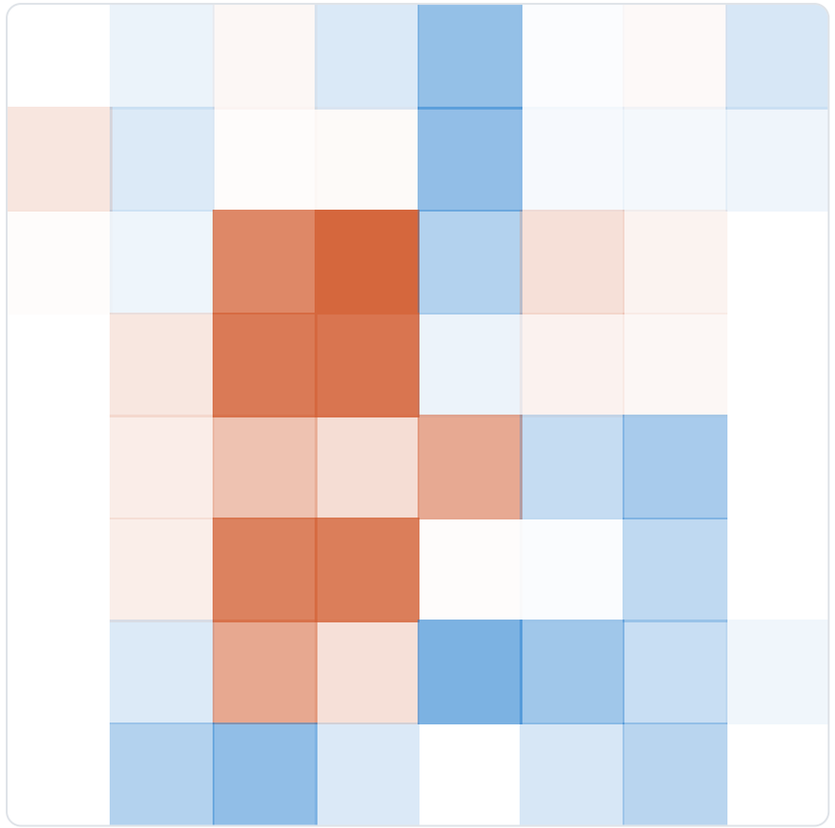
ответ: **8**
z = 8,4
p(8) = **1,000**
верно

357 изображений: **214 train · 71 validation · 72 test**, стратифицированное разбиение. Модель: `StandardScaler` + `LogisticRegression`. На рисунке — восемь изображений validation, решение при пороге 0,5.

Третья слева цифра: score 0,81 → p(8) = 0,693. Модель ошиблась, но и уверенность невысокая. У остальных семи ответ верный.

Данные: `sklearn.datasets.load_digits` — тестовая часть UCI Optical Recognition of Handwritten Digits.

Веса логрега можно нарисовать



- вес > 0: яркий пиксель повышает score «8»
- вес < 0: повышает score «3»

64 веса — по одному на пиксель. Разложим их в ту же сетку 8×8.

Оранжевые пиксели — в левой половине. У восьмёрки там левые стороны петель, у тройки пусто: яркий пиксель в этом месте повышает score «8».

> Score изображения — сумма «яркость пикселя × вес». Ровно та же сумма вкладов, что в прогнозе для Dodge Aspen.

Веса показаны для стандартизованных признаков. Это описание модели, а не «объяснение» того, как устроены цифры.

Без штрафа веса логрега растут

Если классы на train разделяются прямой, модели выгодно увеличивать веса: вероятности уходят к 0 и 1, log loss на train стремится к нулю.

C	Длина вектора весов	Log loss train	Log loss validation
0,01	0,69	0,203	0,210
0,1	1,57	0,060	0,084
1	2,93	0,013	0,043
10	4,71	0,002	0,033
100	6,64	0,0004	0,037
10 000	7,62	0,0002	0,038

`C` — **обратная** сила штрафа: больше `C` — слабее ограничение.

Train-ошибка падает всё время. Validation — лучшая при `C = 10`, дальше модель становится самоуверенее и чуть хуже.

Digits, 3 против 8. В `LogisticRegression` L2-штраф включён по умолчанию, `C = 1`.

Три вопроса к классификатору — три метрики

Вопрос	Метрика	Что подаём на вход
Хороши ли сами вероятности?	**Log loss ↓**	вероятности
Правильно ли модель упорядочивает объекты?	**ROC-AUC ↑**	вероятности или score
Хороши ли решения после порога?	**Precision, recall, F1 ↑**	метки классов

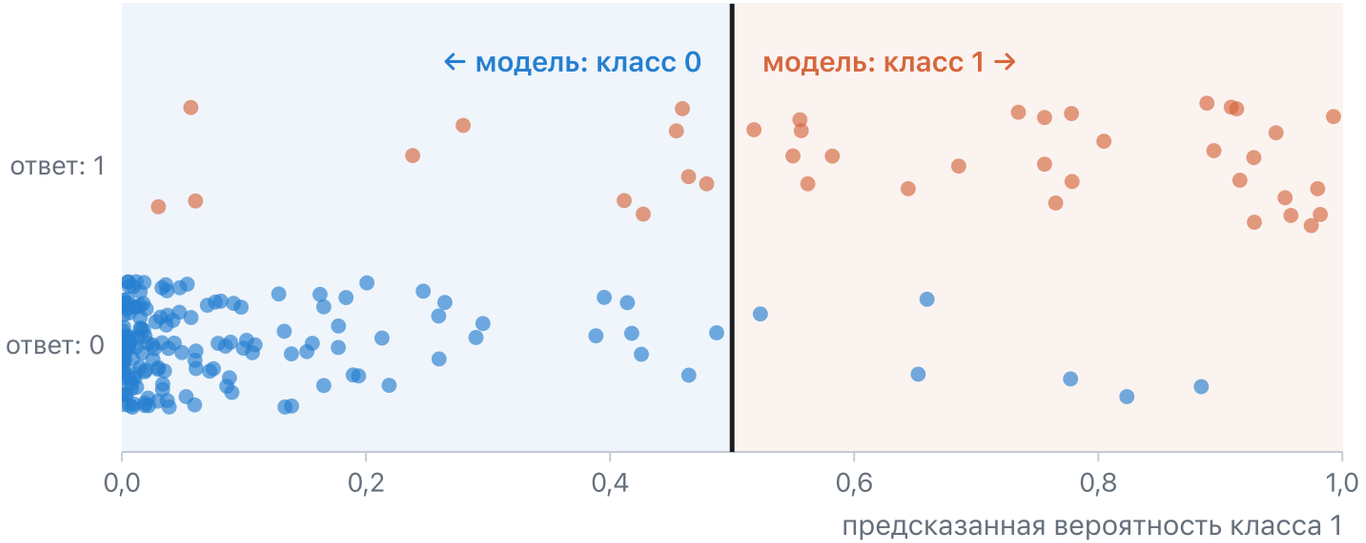
Пример. Сделаем модель самоувереннее: умножим все score на 3. Порядок объектов не изменится.

Ответ y	0	1	0	1	ROC-AUC	Log loss
Вероятности p	0,10	0,40	0,60	0,90	0,75	0,511
После умножения score на 3	0,001	0,229	0,771	0,999	0,75	0,739

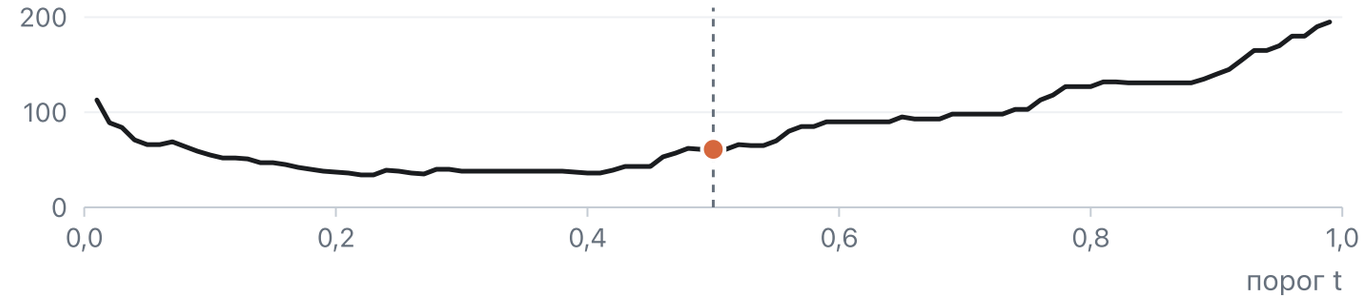
Одинаковый ROC-AUC не означает одинаковые вероятности. Модель стала увереннее и там, где ошибается.

Порог выбираем под задачу

Порог t: класс 1, если p ≥ t 0,50 | t = 0,5 | минимум стоимости: 0,23 | максимум F1: 0,42



Стоимость ошибок: FP + 5·FN



	модель: 1	модель: 0
ответ: 1	TP 29	FN 11
ответ: 0	FP 6	TN 154

Precision	0,829
Recall	0,725
F1	0,773
Стоимость FP + 5·FN	61
ROC-AUC (не зависит от порога)	0,948

Учебная синтетика: два нормальных облака, доля класса 1 около 20%, логрег обучен на 600 объектах, показаны 200 объектов validation. При равной стоимости выбираем больший порог.

БЫСТРЫЙ ОПРОС

Логистическая регрессия выдала для объекта p = 0,3. Какой класс предскажет модель?

A Класс 0: вероятность меньше половины

B Класс 1: вероятность больше нуля

✓ Зависит от порога: при t = 0,5 — класс 0, при t = 0,2 — класс 1

D Никакой: p = 0,3 означает, что модель не обучилась

Вероятность и класс — разные величины. Метод predict использует порог 0,5, но это соглашение по умолчанию, а не свойство модели. Порог выбирают под цену ошибок.

БЫСТРЫЙ ОПРОС

В LogisticRegression увеличили C с 1 до 1000. Что произойдёт?

✓ Штраф ослабнет: веса вырастут, log loss на train уменьшится

B Штраф усилится: веса станут ближе к нулю

C Изменится порог классификации

D Модель превратится в линейную регрессию

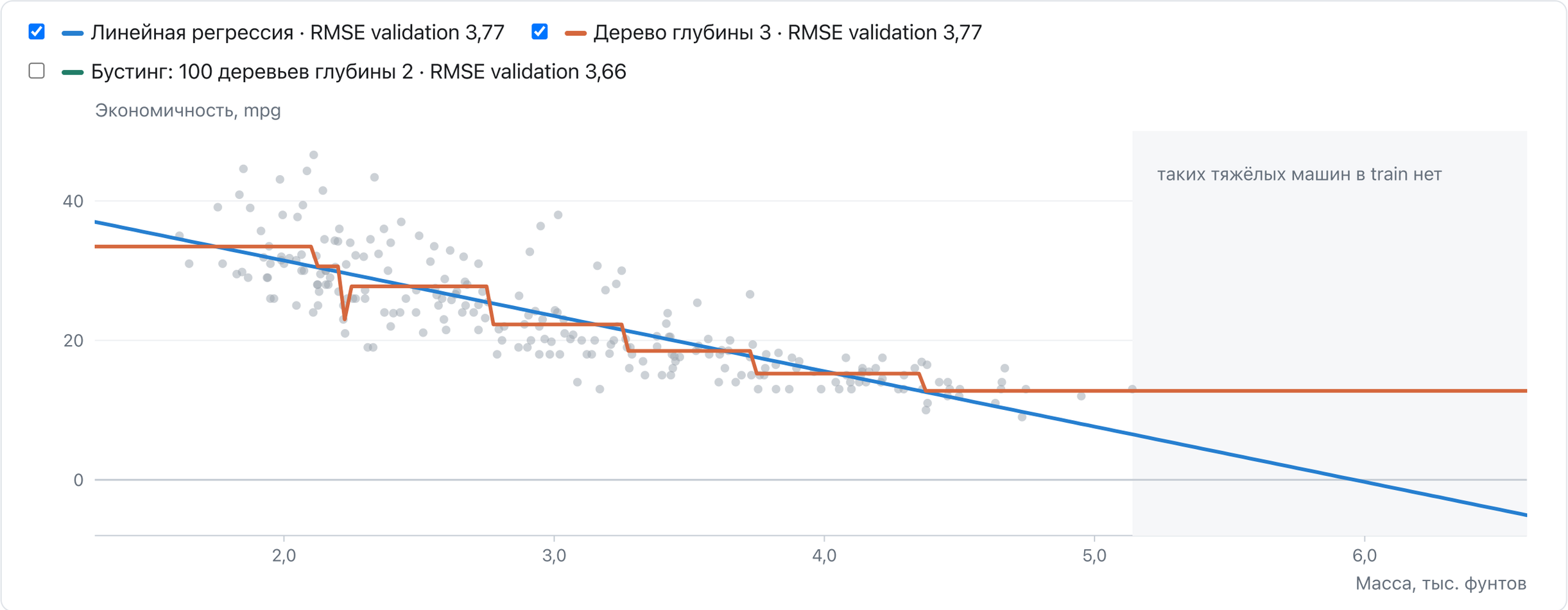
C — обратная сила регуляризации. Большое C разрешает большие веса. Качество на validation при этом может как вырасти, так и упасть — проверяем.

ЧАСТЬ 4

Собираем всё вместе

Деревья, бустинг, линейная и логистическая регрессия: чем они отличаются и с чего начинать новую задачу.

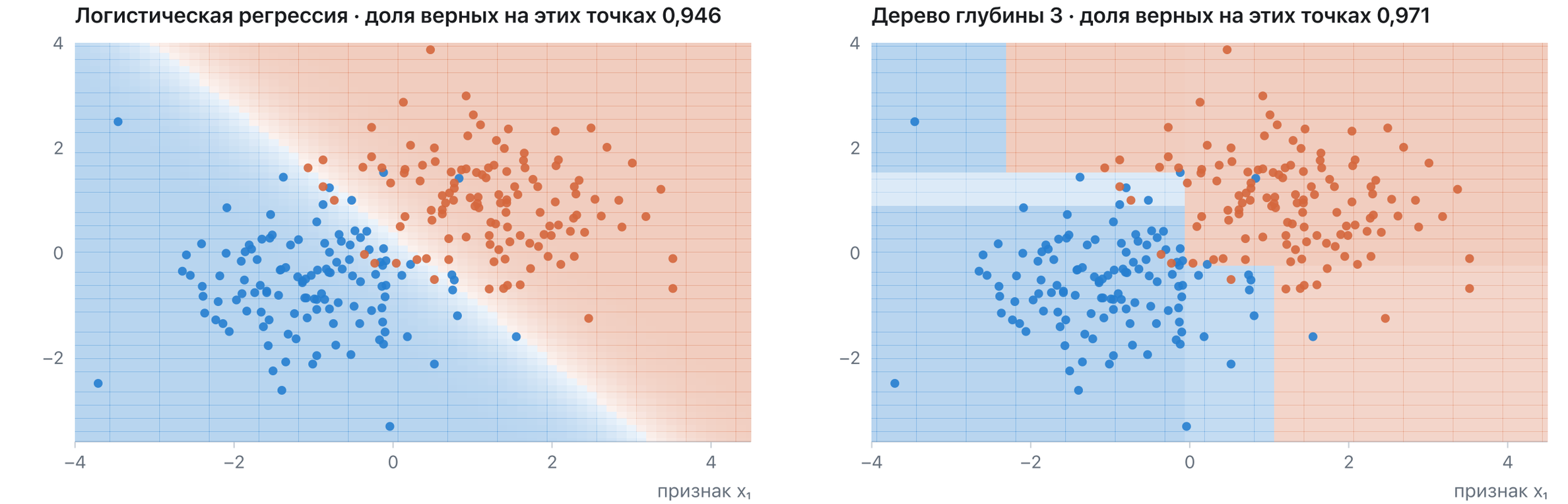
Дерево и прямая на одних данных



Внутри данных качество близкое. Дерево рисует ступеньки, прямая — общий тренд.

За границей train дерево повторяет ответ крайнего листа. Прямая продолжает тренд и уходит ниже нуля — экономичность не бывает отрицательной.

Граница классов: логрег и дерево



Учебная синтетика: два нормальных облака по 120 точек, seed = 1. Цвет фона — предсказанная вероятность класса 1; обе модели обучены на показанных точках.

Логрег проводит одну прямую и плавно меняет вероятность. Дерево режет плоскость на прямоугольники с постоянным ответом.

Дерево, линрег и логрег: сравнение

	Дерево, бустинг	Линейная регрессия	Логистическая регрессия
Задача	регрессия и классификация	регрессия	классификация
Что выдаёт	ответ листа или сумму листьев	число $w \cdot x + b$	вероятность $\sigma(w \cdot x + b)$
Форма	ступеньки, прямоугольники	прямая, плоскость	прямая граница классов
Что обучаем	признаки и пороги в узлах	веса: МНК или спуск	веса: спуск по log loss
Взаимодействия	находит само при глубине ≥ 2	нужен столбец $x_1 x_2$	
Масштаб признаков	не важен	важен для спуска и штрафа	
Вне диапазона train	константа	продолжает тренд	p уходит к 0 или 1
Защита от переобучения	глубина, шаг, число деревьев	L1- и L2-штраф, отбор признаков	
Как заглянуть внутрь	важности, SHAP, ICE	веса — при одинаковом масштабе и с оглядкой на похожие признаки	

Зачем линейные модели, если есть бустинг

Быстрый ориентир

Обучается за секунды, почти без настроек. Сразу видно, насколько задача вообще решаема.

Если сложная модель не обгоняет простую — повод проверить признаки, данные и схему проверки.

Много разреженных признаков

Тексты как «мешок слов», one-hot с тысячами категорий. Здесь линейные модели часто на уровне сложных.

Плюс L1-штраф сам отберёт признаки.

Тренд и мало данных

Когда зависимость близка к прямой, а объектов немного, две-три величины оценить проще, чем сотни листьев.

И прогноз продолжается за пределы train — если тренду можно верить.

Кроме того, линейный score внутри: у каждого нейрона нейросети, в стекинге поверх других моделей, в самой логистической регрессии.

ФИНАЛЬНЫЙ ОПРОС · 1 ИЗ 3

В test попалась машина тяжелее любой из train. Как поведут себя модели «масса → экономичность»?

A Обе выдадут среднее по train

✓ Дерево выдаст ответ крайнего листа, прямая продолжит снижать прогноз

C Дерево продолжит тренд, прямая выдаст константу

D Обе откажутся делать прогноз

Дерево не знает ничего за последним порогом — там тот же лист. Линейная модель экстраполирует; хорошо это или плохо, зависит от того, продолжается ли тренд на самом деле.

ФИНАЛЬНЫЙ ОПРОС · 2 ИЗ 3

Доход — в рублях, возраст — в годах. Признаки не масштабировали. Какой модели это безразлично?

A kNN

B Ridge

C Логрегу, обучаемому градиентным спуском

✓ Градиентному бустингу на деревьях

Дерево сравнивает признак с порогом — от единиц результат не зависит. kNN считает расстояния, Ridge штрафует веса, спуску нужен общий шаг: всем трём масштаб важен.

ФИНАЛЬНЫЙ ОПРОС · 3 ИЗ 3

Скидка повышает продажи только в выходные. В таблице есть столбцы «скидка» и «выходной». Какая модель уловит это без новых признаков?

- ✓ Бустинг с деревьями глубины 2
- B Линейная регрессия
- C Ridge с правильно подобранным alpha
- D Никакая: нужны другие данные

Дерево глубины 2 задаёт два вопроса подряд: «выходной?» и «есть скидка?». Линейной модели нужен столбец «скидка × выходной» — тогда и она справится.

Вы открыли новую задачу. С чего начнёте?

Три постановки. Для каждой назовите первые три действия.

ЗАДАЧА A

Цена квартиры. 500 объектов, 12 числовых признаков. Метрика — RMSE.

Что предсказывать «в лоб»? Какую модель первой?

ЗАДАЧА B

Уйдёт ли клиент. 20 000 строк, 5% ушедших, есть категории и пропуски. Метрика — ROC-AUC.

Что сдавать: метки или вероятности?

ЗАДАЧА C

Спам или нет. Каждое письмо — 20 000 столбцов «встречается ли слово». Метрика — F1.

Что выбрать после обучения модели?

Порядок действий при виде задачи

01
Задача и метрика
Число или класс? Что считается ошибкой и что сдаём: метки или вероятности?

02
Разбиение
Train, validation, test — до любой подгонки. Для классов — стратификация.

03
Константа
Среднее, медиана или частый класс. Точка отсчёта для всех моделей.

04
Признаки
Пропуски, one-hot, масштаб. Всё внутри Pipeline, обучение только на train.

05
Простая модель
Ridge или логрег. Быстро и видно, решаема ли задача.

06
Настройка
alpha или C, новые признаки, порог — по validation.

07
Сильная модель
Бустинг. Сравниваем по той же схеме и той же метрике.

08
Финал
Test — один раз. Записать, что пробовали и что получилось.

Спасибо за внимание!

Вопросы?

